# Course Outline: Managing Database Evolution with FastAPI

**Title:** Managing Database Evolution with FastAPI **Skill:** Skill 33 — Relacionando objetos y tablas utilizando un ORM **Learnpack:** + Gestionando la evolución de la base de datos con FastAPI **File:** s33-w12d34-gestionando-la-evolucion-de-la-base-de-d **Prerequisite:** Object-Relational Mapping: El traductor universal **Target Audience:** Beginner developers who have defined SQLAlchemy models and understand ORM basics **Total Lessons:** 9 **Format:** Mixed (11% hands-on)

---

## Course Description

Your SQLAlchemy models are defined, your ORM is working — but your database schema is frozen at the moment you first ran `Base.metadata.create_all()`. Every time your app evolves, you risk either manually patching the database or dropping and recreating everything. Migrations solve this problem. In this course, students learn what database migrations are, why they matter, how to use Alembic with FastAPI to manage schema changes safely, and when migrations are the right tool — and when they're not.

---

## Course Philosophy

This course emphasizes:

- Schema evolution as a normal, expected part of development — not an emergency
- Using the right tool (Alembic) rather than workarounds that break in production
- Understanding the decision: migrations add overhead, so knowing when to skip them matters as much as knowing how to use them

---

## Course Statistics Summary

| Section | Lessons |
|---|---|
| 00 - Welcome | 1 |
| 01 - Understanding Migrations | 2 |
| 02 - Alembic with FastAPI | 4 |
| 03 - Assessment | 1 |
| 04 - Outro | 1 |
| **Total** | **9** |

---

00.0 Your App Is Live. Now What? 📖

**Learning Objectives:**

- Identify the schema evolution problem that migrations solve
- Understand what this course will cover and how it connects to their ORM knowledge

**Content Outline:**

- The frozen schema problem: once `create_all()` has run in production, adding columns or tables is no longer safe to do manually
- What happens when you change a model but not the database: mismatched state, data loss risk, server crashes
- The solution: a versioned, reversible system for changing your schema — migrations
- What students will be able to do by the end: create, run, and roll back migrations in a FastAPI project using Alembic
- Course sections preview: what migrations are → Alembic in FastAPI → when to use them

**Transitions:** Leads directly into Section 01, where we define what migrations are and why they exist.

**Key Principles:**

- Production databases cannot simply be dropped and recreated — data must be preserved
- Schema changes need to be tracked, versioned, and reversible, just like code changes

**Examples:**

- A working user table in production, then a new feature requires adding a `phone_number` column — how do you do that without losing existing data or downtime?
- A team of 3 developers all adding columns to the same table in different branches — how do they stay in sync?

---

## 01.0 Migrations 🕮

**Learning Objectives:**

- Define what a database migration is
- Explain how migrations differ from calling `create_all()` at startup
- Describe the migration lifecycle: create → apply → rollback

**Content Outline:**

- What a migration is: a versioned, incremental script that transforms the database schema from one state to another
- How it differs from `Base.metadata.create_all()`: `create_all()` only creates tables if they don't exist — it cannot add or remove columns, change types, or rename tables. Migrations do all of this safely.
- The migration lifecycle:
  - `upgrade`: move the schema forward (add a column, create a table)
  - `downgrade`: move the schema backward (undo the change)
  - Each migration has a unique ID and records what came before it

- The migration history: a chain of numbered snapshots of the schema, stored as files in your project

**Transitions:** Coming from ORM basics (models, sessions) — students have already seen how SQLAlchemy represents schema in Python. Migrations are the mechanism that keeps the actual database in sync with those models over time. Leads into the next lesson on advantages and decision criteria.

**Key Principles:**

- Migrations are small, reversible steps — never one giant irreversible jump
- Each migration is a script with an `upgrade` function and a `downgrade` function
- The migration history is stored alongside your code, versioned in git

**Examples:**

- Analogy: like git commits for your database schema. Each commit moves forward; you can check out an older state if something goes wrong.
- Real scenario: add an `is_verified` column to the users table without touching existing rows

---

## 01.1 Advantages and Trade-offs 🕮

**Learning Objectives:**

- List the key advantages of using migrations in a production project
- Identify scenarios where migrations are the right choice
- Identify scenarios where migrations add unnecessary overhead

**Content Outline:**

- **Advantages:**
  - Safe evolution: apply changes without losing data
  - Team coordination: every developer applies the same changes in the same order — no more "works on my machine" schema mismatches
  - Rollback capability: if a migration breaks something, downgrade brings the schema back to the previous state
  - Audit trail: every schema change is documented, timestamped, and reviewable in git history
  - CI/CD compatibility: migrations run as part of the deployment pipeline, keeping staging and production in sync automatically
- **When to use migrations:**
  - Any project with a production database (real users, real data)
  - Team projects where multiple developers share a database schema
  - Projects using CI/CD pipelines
  - Anywhere rollback capability matters
- **When NOT to use migrations:**
  - Early prototyping where the schema changes hourly and no real data exists (use `create_all()` with `drop_all()` freely)

    - Throw-away scripts or one-off tools with no persistent data
    - When you're the only developer on a local-only project with no real data — the overhead isn't worth it yet

**Anti-pattern — Running raw SQL in production instead of migrations:**

- Tempting because it feels "simpler" or "faster"
- What goes wrong: no history of the change, no way to roll back, other developers don't know about it, staging and production drift apart
- The correct approach: always use a migration, even for "small" changes

**Transitions:** Now that the concept and decision framework are clear, the next section moves to the tool that makes all of this happen in FastAPI: Alembic.

**Key Principles:**

- Migrations are worth the setup cost as soon as real data exists
- "One-time" manual SQL changes compound — use migrations to prevent schema drift

**Examples:**

- Project A (early startup): schema changes 5 times a day in week 1. Using `drop_all()/create_all()` is fine here.
- Project B (6 months in, 500 users): a feature requires a new column. Dropping the table is not an option — migration required.

---

## 02.0 Alembic 🕮

**Learning Objectives:**

- Identify what Alembic is and why it's the standard migration tool for SQLAlchemy
- Understand how Alembic fits into a FastAPI project alongside existing SQLAlchemy models

**Content Outline:**

- What Alembic is: the official migration tool for SQLAlchemy. It reads your SQLAlchemy models, compares them to the current database state, and generates migration scripts automatically.
- Where Alembic fits in the FastAPI stack: alongside SQLAlchemy models and the database engine — it doesn't replace anything, it sits on top
- What Alembic produces: a `migrations/` folder with a version history of all schema changes, and an `alembic.ini` configuration file
- The two ways to create migrations:
    - `--autogenerate`: Alembic compares your models to the DB and generates the script automatically (most common)
    - Manual: write the `upgrade`/`downgrade` functions yourself (for data migrations or complex changes autogenerate can't detect)
- Section preview: next lessons cover setup → creating migrations → running and rolling back → hands-on practice

**Transitions:** Leads into the setup lesson, which walks through installing Alembic and wiring it to an existing FastAPI+SQLAlchemy project.

**Key Principles:**

- Alembic is to database schemas what git is to source code
- `--autogenerate` handles most structural migrations; manual migrations handle data transformations

**Examples:**

- A FastAPI project with a `User` model. You add a `phone` column to the model. Alembic detects the difference and generates the migration script automatically.

---

## 02.1 Setup and Configuration 🕮

**Learning Objectives:**

- Install Alembic in a FastAPI project
- Configure Alembic to connect to the database and find SQLAlchemy models
- Understand the purpose of `alembic.ini` and `env.py`

**Content Outline:**

- Install: `pip install alembic`
- Initialize: `alembic init migrations` — creates the `migrations/` folder and `alembic.ini`
- **`alembic.ini`:** The main config file. Key setting: `sqlalchemy.url = postgresql://user:pass@host/db` (or use an environment variable)
- **`migrations/env.py`:** The script Alembic runs to connect to the database. Two critical changes required:
    1. Import your `Base` (declarative base): `from app.models import Base`
    2. Set `target_metadata = Base.metadata` — this tells Alembic what your models look like
- Why `env.py` matters: without `target_metadata`, `--autogenerate` cannot compare your models to the database — it will generate empty migrations

**Anti-pattern — Hardcoding the database URL in `alembic.ini`:**

- Tempting because it's the simplest setup
- What goes wrong: the URL with credentials ends up in version control, a security risk
- The correct approach: read the URL from an environment variable in `env.py`: `config.set_main_option("sqlalchemy.url", os.environ["DATABASE_URL"])`

**Transitions:** With Alembic configured and aware of your models, you're ready to create your first migration.

**Key Principles:**

- `env.py` is the bridge between Alembic and your SQLAlchemy models
- Configuration changes are one-time — once set up, creating migrations is just one command

**Examples:**

- A FastAPI project structure:

```
app/
  models.py   ← SQLAlchemy models and Base
  database.py ← engine and session
migrations/   ← created by alembic init
  env.py      ← configured to import Base from app.models
alembic.ini   ← database URL via env variable
```

---

## 02.2 Creating and Running Migrations 🕮

**Learning Objectives:**

- Generate a migration automatically from model changes using `--autogenerate`
- Inspect a generated migration file before applying it
- Apply a migration with `alembic upgrade head`
- Roll back a migration with `alembic downgrade -1`
- Check migration history with `alembic history` and current state with `alembic current`

**Content Outline:**

- **Creating a migration:**
  - `alembic revision --autogenerate -m "add phone to users"` — generates a new file in `migrations/versions/`
  - The generated file contains two functions: `upgrade()` and `downgrade()`
  - `upgrade()` example: `op.add_column('users', sa.Column('phone', sa.String(20)))`
  - `downgrade()` example: `op.drop_column('users', 'phone')`
  - Always inspect the generated file before applying — autogenerate is not perfect. It cannot detect column renames or data transformations.
- **Running a migration:**
  - `alembic upgrade head` — applies all pending migrations up to the latest
  - `alembic upgrade +1` — applies exactly one migration forward
- **Rolling back:**
  - `alembic downgrade -1` — reverts the most recent migration
  - `alembic downgrade base` — reverts all migrations back to an empty schema
- **Checking status:**
  - `alembic history` — lists all migrations in order with their IDs and messages
  - `alembic current` — shows which migration the database is currently at

**Anti-pattern — Applying without inspecting:**

- Tempting to trust `--autogenerate` blindly

- What goes wrong: autogenerate misses some changes (renames, indexes not declared in models, data migrations). Applying an incomplete migration leaves the schema inconsistent.
- The correct approach: always open and read the generated migration file before running `alembic upgrade`

**Transitions:** Now that you understand the full migration workflow, the next lesson gives you hands-on practice writing and running a migration.

**Key Principles:**

- `upgrade head` is the deployment command — run it when deploying to apply pending migrations
- `downgrade -1` is the emergency rollback — revert the last migration if something breaks in production
- Each migration file is a Python script you can read, edit, and version in git

**Examples:**

- Full workflow: add `is_admin = Column(Boolean, default=False)` to User model → run `alembic revision --autogenerate -m "add is_admin"` → inspect the generated file → run `alembic upgrade head` → verify the column exists in the database

---

## 02.3 Write a Migration 💻

**Challenge Prompt:** Add a `bio` column (nullable Text) to an existing `User` model, then generate and apply an Alembic migration that adds that column to the database.

**Solution Code:**

```
# app/models.py — add bio column to User model
from sqlalchemy import Column, Integer, String, Text
from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    username = Column(String(80), nullable=False)
    bio = Column(Text, nullable=True)  # new column added
```

```
# migrations/versions/xxxx_add_bio_to_users.py — generated by alembic
revision --autogenerate
def upgrade():
    op.add_column('users', sa.Column('bio', sa.Text(), nullable=True))
```

```
def downgrade():
    op.drop_column('users', 'bio')
```

**Challenge Code:**

```
# app/models.py — add a bio column here
from sqlalchemy import Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    username = Column(String(80), nullable=False)
    # TODO: add a nullable Text column called bio

# After editing this file:
# 1. Run: alembic revision --autogenerate -m "add bio to users"
# 2. Inspect the generated migration file
# 3. Run: alembic upgrade head
```

---

## 03.0 What You Know Now 🤔

**Learning Objectives:**

- Recall what database migrations are and how they differ from `create_all()`
- Identify the advantages of migrations in production and team projects
- Describe the Alembic setup process and the role of `env.py`
- Trace the migration workflow from model change to applied schema change
- Apply the decision framework: when to use migrations and when to skip them

**Question Format:** Scenario-based

**Questions:**

1. Your FastAPI app has been running in production for 3 months with 2,000 users. You need to add a `last_login` timestamp column to the `User` table. Which approach is correct? a) Drop the `users` table and recreate it with the new column b) Run `Base.metadata.create_all()` — it will add the missing column c) Create an Alembic migration, inspect it, and run `alembic upgrade head` d) Connect to the database and run `ALTER TABLE users ADD COLUMN` manually

2. You run `alembic revision --autogenerate -m "add column"` and the generated file's `upgrade()` function is empty. What is the most likely cause? a) The database is already up to date b) `target_metadata` in `env.py` is not set to `Base.metadata` — Alembic doesn't know about your models c) You forgot to install Alembic d) The migration has already been applied

3. A junior developer on your team says: "I just manually ran `ALTER TABLE users ADD COLUMN phone VARCHAR(20)` in the production database. Faster than writing a migration!" What problem does this create? a) None — manual SQL is equivalent to a migration b) The change exists in production but not in the migration history, so other developers, staging, and CI/CD won't have it c) The column will be deleted the next time `create_all()` runs d) Alembic will automatically detect and undo the change

4. You want to undo the migration you just applied because it caused an error. Which command do you run? a) `alembic upgrade -1` b) `alembic reset` c) `alembic downgrade -1` d) `alembic rollback head`

5. A colleague asks: "We're building a quick proof of concept this week — the schema will change 10 times before we decide on the final structure. Should I set up Alembic now?" What is the correct advice? a) Yes, always use Alembic from day one regardless of the project stage b) No — during rapid prototyping with no real data, using `drop_all()/create_all()` freely is fine. Set up Alembic when the schema stabilizes and real data enters the picture. c) Use Alembic but don't commit the migration files to git yet d) Use raw SQL ALTER TABLE statements during prototyping, then switch to Alembic later

6. Which file must you edit after running `alembic init migrations` to make `--autogenerate` work correctly? a) `alembic.ini` — set the correct `sqlalchemy.url` b) `migrations/env.py` — import your `Base` and set `target_metadata = Base.metadata` c) Both `alembic.ini` and `migrations/env.py` d) Neither — autogenerate works automatically after `alembic init`

**Evaluation Criteria:**

- 6/6: Full mastery — ready to own migrations in a production project
- 4-5/6: Solid understanding with minor gaps — review the scenarios missed
- 2-3/6: Partial understanding — revisit sections 01 and 02
- 0-1/6: Foundational review needed — start from the beginning

**Score Interpretation:** Migrations are a professional skill that prevents data loss and team friction. 5+ correct means you're ready to apply this in real projects.

---

## 04.0 Outro

**Content Outline:**

- Course recap: what students accomplished
  - Learned what database migrations are and how they differ from simply calling `create_all()`
  - Understood the advantages: safe schema evolution, team coordination, rollback capability, audit trail
  - Set up Alembic in a FastAPI project (init, env.py, alembic.ini)
  - Ran the full migration workflow: model change → autogenerate → inspect → upgrade → downgrade
  - Applied the decision framework: when migrations are worth the setup, and when they're not

- Connection to bigger picture: migrations are one of the pillars of production-grade FastAPI development. Together with SQLAlchemy models and sessions (previous learnpack), you now have the complete backend data layer under control.
- Next steps: containerizing your FastAPI app with Docker is the natural next step — and migrations integrate cleanly into a Docker deployment pipeline
- Resources for further exploration:
    - Alembic documentation: https://alembic.sqlalchemy.org/
    - FastAPI + SQLAlchemy full example in the FastAPI docs
- Encouragement: Production databases are serious business — knowing how to evolve them safely is a skill that separates junior developers from professional ones.

**Note:** This is conclusion only — no theory, exercises, or assessment.

---